

Hybrid Greedy-Dynamic Programming Approach for Multi-Criteria Weighted Activity Selection

John Cyril Espina, Glenn Mark Anino*, Den Jester Antonio, Junar Landicho

Department of Information Technology and Computing (CITC), University of Science and Technology of Southern Philippines (USTP),
Cagayan de Oro City, Philippines

ARTICLE INFO

Keywords:

Multi-criteria scheduling
Activity selection problem
Fairness optimization
Task allocation
Hybrid algorithm design
Resource-constrained Scheduling

ABSTRACT

The Multi-Person Activity Selection Problem and Weighted Activity Selection Problem extend the traditional Activity Selection Problem to complex real-world scheduling scenarios. Recent studies (2021–2024) have advanced these domains separately, but integration challenges persist. This study proposes a hybrid algorithm integrating greedy heuristics with dynamic programming to enhance fairness among participants while maximizing total value in multi-criteria weighted activity selection. The algorithm operates in three phases: initial greedy allocation for efficiency, dynamic programming refinement for optimality, and fairness optimization to ensure equitable distribution. The approach was evaluated on synthetic datasets (50-500 activities), benchmark instances, and real-world case studies including hospital scheduling, conference management, and software project allocation. Results demonstrate significant performance improvements over pure approaches. The hybrid algorithm achieved 42-48% assignment rates across datasets while maintaining high fairness scores (Jain's index > 0.94). Compared to ScheduleNet benchmark, it delivered 10x-23x better makespan optimization with runtime under 100ms even for large-scale problems. Real-world applications showed consistent value improvements: 101% in healthcare resource allocation, 65% in conference scheduling, and 32% in project management, while achieving final fairness scores of 0.64-0.88. The algorithm addresses limitations in recent methodologies, particularly integration challenges between fairness and optimization objectives. This hybrid approach contributes to more adaptable scheduling solutions that simultaneously satisfy fairness requirements and optimization goals, providing a practical framework for automated resource allocation in multi-agent systems with sub-second computational efficiency suitable for real-time applications.

1 INTRODUCTION

The Activity Selection Problem (ASP) has long been a fundamental topic in scheduling and optimization, typically solved using greedy algorithms that efficiently identify the maximum number of non-overlapping activities (Cormen et al., 2009). However, traditional solutions fall short in addressing the complexities of real-world applications, where multiple decision-makers and differing priorities add significant challenges. Two prominent extensions have emerged to address these limitations: the Multi-Person Activity Selection Problem and the Weighted Activity Selection Problem (WASP).

The Multi-Person Activity Selection Problem expands the classic model by considering multiple participants making activity choices simultaneously, introducing complications such as fairness, optimal resource allocation, and participant preferences. Liu et al. (2023) highlighted how traditional scheduling techniques often struggle under these multi-agent conditions. Similarly, Braquet and Bakolas (2021) explored fairness and efficiency in distributed task assignments, while Park et al. (2021) emphasized the difficulty of balancing equity with system-wide efficiency.

On the other hand, the Weighted Activity Selection Problem assigns different weights or values to each activity, shifting the objective from simply maximizing the number of activities to maximizing the total value derived. Shah et al. (2024) addressed this challenge in the context of multi-person activity recognition with efficiency constraints, and Liu et al. (2024) advanced the field by employing submodular optimization for distributed task allocation. Despite these advances, existing methods show critical shortcomings in achieving fairness among participants, optimizing value in selection, and combining multiple criteria in a unified approach.

*Corresponding author.

Email addresses: glennmark.anino@1.ustp.edu.ph (G.M. Anino), denjester.antonio@1.ustp.edu.ph (D.J. Antonio), johnncyril.espina@1.ustp.edu.ph (J.C. Espina), junarlandicho@ustp.edu.ph (J. Landicho).

Many current solutions either prioritize fairness at the expense of overall value or focus on value maximization while neglecting equitable distribution. Moreover, greedy-based methods often suffer from local optimality, failing to consider the global impact of individual selections. Although recent studies have made progress in addressing either multi-participant or weighted selection challenges, there is a lack of integrated approaches that reconcile both fairness and optimality in a scalable and computationally efficient way

This study seeks to fill that gap by proposing a hybrid algorithm that combines greedy heuristics with dynamic programming refinement. The goal is to achieve fair and optimal selection in multi-participant, weighted activity selection scenarios. The algorithm is designed to equitably allocate non-overlapping, indivisible activities to multiple participants with varying constraints, while maximizing the total weighted value of selected tasks. The approach will be validated using both synthetic and real-world datasets, providing a comprehensive assessment of improvements in efficiency, fairness, and overall selection quality.

2 RELATED STUDIES

The field of multi-person scheduling and weighted activity selection has witnessed significant progress, though researchers have consistently encountered the challenge of balancing fairness with optimization efficiency. [Liu et al. \(2023\)](#) established foundational understanding by demonstrating that traditional single-agent approaches fail when naively applied to multi-person scenarios, introducing a deep multi-agent reinforcement learning approach with centralized training and decentralized execution that balances individual priorities with system-wide goals. Building on this, [Park et al. \(2021\)](#) conceptualized scheduling as an agent-task graph problem through ScheduleNet, employing type-aware graph attention mechanisms while establishing theoretical bounds for greedy and dynamic programming approaches in complex scheduling scenarios. The trajectory shifted toward decentralized solutions with [Braquet and Bakolas \(2021\)](#), whose Greedy Coalition Auction Algorithm enables agent autonomy through utility-based bidding while simultaneously considering energy consumption, satisfaction, and spatial constraints to ensure conflict-free assignments. However, [Liu et al. \(2024\)](#) revealed limitations of static approaches through their Distributed Greedy Bundles Algorithm, demonstrating that static weighting schemes fail to capture intricate activity interdependencies, leading them to develop adaptive weighting mechanisms focused primarily on resource constraints rather than fairness considerations. Domain-specific applications like [Shah et al.'s \(2024\)](#) lightweight vision-based multi-person activity recognition system for robot-assisted healthcare using Multilayer LSTM networks highlight a common pattern where specialized solutions excel in intended contexts but struggle with cross-domain generalization. While each contribution has advanced either fairness or optimization aspects, the simultaneous integration of both dimensions remains an open challenge, representing the gap our proposed hybrid algorithm addresses by combining greedy heuristics with dynamic programming refinement to satisfy both efficiency and fairness requirements that previous work has tackled separately.

3 METHODOLOGY

3.1 Preliminary Definitions

Before formalizing the problem and proposing our algorithm, we introduce key terms and concepts used throughout the methodology. Two activities, a_i and a_j , are considered compatible if their scheduled times do not overlap; formally, their time intervals $[s_i, f_i]$ and $[s_j, f_j]$ have no intersection, where s_i and f_i denote the start and finish times of activity a_i , respectively. To address fairness among participants, we define a fairness tolerance parameter ϵ , which specifies the maximum allowable deviation from an equal distribution of total assigned value. This parameter helps balance the trade-off between maximizing overall value and maintaining fairness across participants. Furthermore, resource constraints are integral to the problem: each activity a_i requires a set of resources R_i , and each participant p_j has a limited capacity C_r for each resource r . A feasible allocation must ensure that the sum of resource demands from all activities assigned to a participant does not exceed their capacities. Finally, the concept of timeline construction is essential for scheduling; a timeline for participant p_j is defined as a sequence of non-overlapping available time slots $T_p = \{[t_1^{start}, t_1^{end}], [t_2^{start}, t_2^{end}], \dots\}$, representing periods when the participant is free and can potentially be assigned additional activities.

3.2 Problem Formulation and Mathematical Model

The Multi-Criteria Weighted Activity Selection Problem involves selecting and allocating a set of activities $A = \{a_1, a_2, \dots, a_n\}$ to a set of participants $P = \{p_1, p_2, \dots, p_m\}$, where each activity a_i is characterized by a start time s_i , finish time f_i , weight w_i , and a set of required resources R_i . Each participant has individual constraints, including resource capacities. The objective is to find an allocation function $\phi : A \rightarrow P \cup \{\emptyset\}$ that maximizes the total weighted value of selected activities, ensures no participant has overlapping activities, maintains fairness among participants, and respects all resource constraints.

To model this problem mathematically, we define decision variables x_{ij} , which equal 1 if activity a_i is assigned to participant p_j , and 0 otherwise, as well as y_i , which is 1 if activity a_i is selected and 0 otherwise.

The optimization problem seeks to maximize the total value:

$$\max \sum_{i=1}^n w_i \cdot y_i$$

Subject to the following constraints:

1. Non-overlapping constraint for each participant:

$\{x_{ij}\} + \{x_{ik}\} \leq 1 \ \forall j \in \{1, 2, \dots, m\}, \forall i, k \in \{1, 2, \dots, n\} \text{ where } [s_i, f_i] \cap [s_k, f_k] \neq \emptyset$ This constraint is implemented in Phase 1 (InitialGreedyAllocation) through time availability tracking (t) for each participant.

2. Assignment constraint ensuring each activity is assigned to at most one participant:

$\sum_{j=1}^m x_{ij} = y_i \ \forall i \in \{1, 2, \dots, n\}$ This is maintained throughout all phases by tracking each activity's assignment status.

3. Fairness constraint ensuring balanced allocation:

$|\sum_{i=1}^n w_i \cdot x_{ij} - \frac{1}{m} \sum_{i=1}^n w_i \cdot y_i| \leq \varepsilon \quad \forall j \in \{1, 2, \dots, m\}$ where ε is a fairness tolerance parameter. This is primarily addressed in Phase 3 (FairnessOptimization).

4. Resource constraints:

$\sum_{i \in R} x_{ij} \leq C_r \quad \forall r \in \text{Resources}, \forall j \in \{1, 2, \dots, m\}$ where C_r is the capacity of resource r . These constraints are verified during assignment in Phase 1 and preserved during reallocations in subsequent phases.

To quantify the fairness of an allocation ϕ , we define a normalized deviation metric:

$$F(\phi) = 1 - \sqrt{\frac{\sum_{j=1}^m (v_j - \text{avg})^2}{m \cdot v_{\text{avg}}^2}}$$

where v_j is the total value assigned to participant p_j , and v_{avg} is the average assigned value. This metric ranges from 0 (maximum unfairness) to 1 (perfect fairness), and the optimization requires that $F(\phi) \geq F_{\min}$, a minimum fairness threshold.

3.3 Proposed Algorithm

The proposed hybrid greedy-dynamic programming approach operates in three sequential phases, each addressing a specific aspect of the multi-criteria weighted activity selection problem:

3.3.1 Phase 1: Initial Greedy Allocation

The first phase generates an initial feasible allocation using a greedy strategy that emphasizes computational efficiency. This allocation serves as a baseline solution to be enhanced in later phases while ensuring feasibility with respect to scheduling constraints.

The key idea in this phase is to prioritize activities that offer high value relative to their duration and to allocate them in a manner that avoids overlaps and promotes fairness. Instead of relying on complex optimization, this phase uses a simple value-based ranking combined with a fairness-aware participant selection process to quickly build an initial solution.

Algorithm 1: InitialGreedyAllocation

Require: Activities A, participants P

Ensure: Initial allocation ϕ

- 1: Sort activities in non-increasing order of weight/finish time ratio (w_i/f_i)
 - 2: Initialize $\phi(a_i) = \emptyset$ for all $i \in \{1, 2, \dots, n\}$
 - 3: For each participant $p_j \in P$:
 - 4: Initialize available time $t_j = 0$
 - 5: For each activity a_i in sorted order:
 - 6: Find participant p_j with minimum total assigned value such that $t_j \leq s_i$
 - 7: If such participant p_j exists:
 - 8: Set $\phi(a_i) = p_j$
 - 9: Update $t_j = f_i$
 - 10: Return ϕ
-

This algorithm begins by ranking all activities using the value-to-finish-time ratio (w_i/f_i), which helps prioritize high-value activities that require less time, with activities having higher ratios handled first. For each such activity, the algorithm identifies participants whose current availability (tracked via t_j) allows for scheduling the activity without overlap, and among these, the participant with the lowest total assigned value so far is chosen, promoting fairness by distributing workload more evenly. Once an activity is assigned, the participant's availability is updated to the activity's finish time, ensuring that no overlapping occurs, which satisfies the non-overlapping constraint (Constraint 1 in Section 3.2), and by choosing the least-loaded participant, the algorithm begins addressing the fairness constraint (Constraint 3). The phase runs in $O(n \log n + nm)$ time, where n is the number of activities and m is the number of participants, with the sorting step contributing $O(n \log n)$ while the participant search per activity costs up to $O(m)$.

3.3.2 Phase 2: Dynamic Programming Refinement

The second phase improves the initial allocation by reorganizing participant schedules using a localized dynamic programming approach. This refinement focuses on increasing the total value of selected activities while maintaining all hard constraints, such as non-overlapping schedules and resource feasibility.

The key innovation in this phase lies in the construction of individual timelines for each participant and the application of the Weighted Interval Scheduling algorithm on available time slots. Rather than treating timeline construction and scheduling as separate algorithms, they are integrated into a unified optimization process for each participant.

Algorithm 2: DynamicProgrammingRefinement

Require: Initial allocation ϕ , Activities A, Participants P

Ensure: Refined allocation ϕ'

- 1: For each participant $p_j \in P$:
 - 2: $A_j \leftarrow \{a_i \in A \mid \phi(a_i) = p_j\}$
 - 3: Sort A_j by start time
-

```

4: Initialize  $T_j \leftarrow \{[0, T]\}$ 
5: For each activity  $a_i \in A_j$ :
6:   For each  $[t\_start, t\_end] \in T_j$ :
7:     If  $t\_start \leq s_i$  and  $t\_end \geq f_i$ :
8:       Remove  $[t\_start, t\_end]$  from  $T_j$ 
9:       If  $t\_start < s_i$ :
10:        Add  $[t\_start, s_i]$  to  $T_j$ 
11:       If  $f_i < t\_end$ :
12:        Add  $[f_i, t\_end]$  to  $T_j$ 
13:       Break
14: For each  $[t\_start, t\_end] \in T_j$ :
15:    $A_i \leftarrow \{a_i \in A \mid \phi(a_i) = \emptyset \text{ and } s_i \geq t\_start \text{ and } f_i \leq t\_end\}$ 
16:   If  $A_i \neq \emptyset$ :
17:     Sort  $A_i$  by finish time
18:     For  $i = 1$  to  $|A_i|$ :
19:       Compute  $p(i) = \max\{j < i \mid a_j \text{ and } a_i \text{ are non-overlapping}\}$ 
20:        $OPT(0) \leftarrow 0$ 
21:       For  $i = 1$  to  $|A_i|$ :
22:          $OPT(i) \leftarrow \max(w_i + OPT(p(i)), OPT(i-1))$ 
23:       Reconstruct optimal subset  $A\_opt$  from  $OPT$ 
24:       For each  $a \in A\_opt$ :
25:          $\phi(a) \leftarrow p_i$ 
26: Return updated allocation  $\phi'$ 

```

This algorithm refines the initial greedy allocation by performing timeline-based optimization for each participant. First, it retrieves the set of activities currently assigned to each participant and sorts them by start time. The algorithm then constructs a timeline of available (non-overlapping) time slots by removing the time occupied by these assigned activities from the total scheduling horizon. If there are gaps before or after an activity, they are preserved as available time segments.

Once the available time slots are identified, the algorithm considers each one individually. It gathers all unassigned activities that can fully fit within a given slot (i.e., their start and finish times fall entirely within the time window). If such activities exist, the algorithm applies the classic Weighted Interval Scheduling dynamic programming method to find the highest-value non-overlapping subset.

The scheduling subroutine works by first sorting candidate activities by finish time. For each activity a_i , it calculates $p(i)$, which is the latest activity that finishes before a_i starts. Using this compatibility information, it computes the optimal value recursively as:

$$OPT(i) = \max(w_i + OPT(p(i)), OPT(i - 1))$$

The OPT array captures the best total value for the first i activities. Once filled, the optimal activity set is reconstructed and assigned to the participant.

This integration of timeline construction and weighted scheduling per time slot allows each participant's schedule to be locally optimized. The divide-and-conquer strategy transforms the global scheduling challenge into smaller, independent subproblems: one per participant per time slot, each solvable optimally. This significantly enhances the quality of the overall allocation while preserving feasibility. The complexity of this phase is $O(m \cdot n^2)$, where m is the number of participants and n the number of activities.

3.3.3 Phase 3: Fairness Optimization

The third and final phase enhances the refined allocation by improving fairness across participants while preserving feasibility and maintaining or improving the total value. This phase directly targets the fairness constraint defined in Section 3.2 by minimizing disparities in value allocation among participants.

The core idea in this phase is to iteratively reassign activities from over-allocated participants to under-allocated ones, provided that such transfers do not violate the scheduling constraints. Unlike previous phases that emphasized efficiency and local optimality, this phase focuses on balancing the distribution of activity value using a fairness-aware reassignment strategy.

Algorithm 3: FairnessOptimization

Require: Refined allocation ϕ' , Activities A , Participants P

Ensure: Fair allocation ϕ''

```

1: Calculate current value distribution  $V = \{v_1, v_2, \dots, v_m\}$  where  $v_j = \sum\{w_i \mid \phi'(a_i) = p_j\}$ 
2: Calculate target fair value  $v_{target} = (1/m) * \sum_j v_j$ 
3: Identify participants  $P_{high} = \{p_j \mid v_j > v_{target} + \epsilon\}$ 
4: Identify participants  $P_{low} = \{p_j \mid v_j < v_{target} - \epsilon\}$ 
5: While  $P_{high}$  and  $P_{low}$  are not empty:
6:   Select  $p_h \in P_{high}$  and  $p_l \in P_{low}$ 
7:   Find activity  $a_i$  where  $\phi'(a_i) = p_h$  such that:
8:     Reassigning to  $p_l$  is feasible (no overlaps)

```

```

9:   Transfer minimizes  $|v_h - v_{target}| + |v_l - v_{target}|$ 
10:  If such activity exists:
11:    Update  $\phi'(a_i) = p_1$ 
12:    Update  $v_h$  and  $v_l$ 
13:    Update  $P_{high}$  and  $P_{low}$ 
14:  Return updated allocation  $\phi$ 
10: Add  $[f_i, t_{end}]$  to  $T_p$ 
11: Return  $T_p$ 

```

This algorithm begins by calculating the total assigned value for each participant, which reflects how much workload (measured in weighted value) each has received, from which a target fair value is computed as the average across all participants. Based on a tolerance ϵ , participants are then grouped as over-allocated (P_{high}) or under-allocated (P_{low}), after which the algorithm performs iterative improvements by selecting a participant with excess value (p_h) and attempting to reassign one of their tasks to a participant with a deficit (p_l). Each candidate reassignment is checked for feasibility to ensure it does not introduce scheduling overlaps, and among feasible candidates, the one that best reduces the deviation from the target value is selected. This reallocation process continues until either balance is achieved (i.e., all participants fall within ϵ of the target) or no beneficial reassignment is possible, with the algorithm guaranteeing that no hard constraint such as activity overlap or invalid assignment is violated throughout. This phase completes the allocation process by explicitly addressing fairness in a structured, optimization-aware manner, with a time complexity of $O(n^2 + m \log m)$, dominated by the search for suitable activity transfers and the maintenance of participant priority sets.

While the Normalized Deviation Metric $F(\phi)$ is used during Phase 3 to guide fairness optimization, we also report Jain's Fairness Index during the evaluation phase. The Normalized Deviation Metric provides an efficient internal measure to iteratively reduce value imbalance among participants until a fairness threshold is reached. In contrast, Jain's Index is a standardized, widely accepted metric in resource allocation literature, allowing for clearer comparative analysis across datasets and algorithms. By employing both, we ensure that fairness is addressed both operationally during allocation and analytically during result evaluation.

3.3.4 Integration Benefits

The integration of these three phases creates a robust approach that balances computational efficiency with optimization quality and fairness. The initial greedy phase provides an efficiency foundation by delivering a quick, reasonable solution that serves as a foundation for further refinement. Subsequently, the dynamic programming phase achieves local optimality by improving this solution through finding the best possible selections within the established framework. Finally, the fairness optimization phase ensures equitable distribution among participants while preserving the value optimization achieved in previous phases.

This hybrid approach addresses the limitations identified in previous research by simultaneously handling both weighted activity selection and multi-participant fairness constraints. The algorithm achieves this with a balanced computational cost, where Phase 1 operates with $O(n \log n + nm)$ complexity dominated by sorting and initial allocation, Phase 2 requires $O(m \cdot n^2)$ complexity dominated by the dynamic programming optimization, and Phase 3 demonstrates $O(n^2 + m \log m)$ complexity dominated by transfer operations. The overall time complexity is $O(m \cdot n^2 + n \log n + m \log m)$. Since the number of participants (m) is typically much smaller than the number of activities (n) in most practical applications, this effectively simplifies to $O(m \cdot n^2)$, which is dominated by the dynamic programming component. This represents a significant improvement over pure dynamic programming approaches, which would require $O(n^m)$ in the worst case, while providing better solutions than pure greedy methods.

Section 3.3.2 provides a detailed breakdown of this analysis, including the step-by-step derivation of these complexity bounds and their implications for algorithm scalability.

3.4 Evaluation Framework and Analysis

3.4.1 Performance Metrics

The hybrid algorithm is evaluated using four comprehensive metrics that measure different aspects of solution quality:

1. Total Value: The sum of weights of all selected activities ($\sum_{i=1}^n w_i \cdot y_i$), measuring the optimization quality of the solution. This metric directly corresponds to the objective function of our mathematical model.
2. Fairness Metrics: We employ two complementary fairness metrics to evaluate the equity of allocation: a. Normalized Deviation Metric (F): As defined in Section 3.2. The metric is primarily used during the execution of the algorithm in Phase 3 to guide the fairness optimization process. b. Jain's Fairness Index (J): For evaluation purposes, we also employ Jain's fairness index, a widely recognized metric in resource allocation literature:

$J(v_1, v_2, \dots, v_m) = \frac{(\sum_{j=1}^m v_j)^2}{m \cdot \sum_{j=1}^m v_j^2}$ This index ranges from $1/m$ (worst case) to 1 (perfect fairness). While mathematically related to our normalized deviation metric F, Jain's index facilitates comparisons with existing literature that commonly uses this measure. The relationship between these metrics is:

$F(\phi) = 1 - \sqrt{1 - J(v_1, v_2, \dots, v_m)}$ Both metrics measure the same concept (fairness of distribution) but with different scaling properties that make them suitable for different purposes in our approach.

3. Computational Efficiency: Running time compared to pure greedy and pure dynamic programming approaches, measuring scalability across varying problem sizes. This is evaluated using both theoretical complexity analysis and empirical runtime measurements.

4. Approximation Ratio: The empirical ratio between the algorithm's solution value and the optimal solution value (where computationally feasible), providing a measure of solution quality. This is calculated as: $R = \text{Value}(\text{Algorithm}) / \text{Value}(\text{Optimal})$. The theoretical foundation for our approximation guarantees follows from [Sviridenko et al.'s \(2024\)](#) work on submodular optimization with bounded curvature.

Each of these metrics captures a different aspect of solution quality, allowing for comprehensive evaluation of the hybrid algorithm against alternative approaches across various problem instances.

3.4.2 Algorithm Analysis

Time Complexity Analysis

The time complexity of each phase is derived as follows:

The algorithm consists of three phases, each with distinct time complexities. Phase 1 (Greedy Allocation) begins by sorting n activities, which takes $O(n \log n)$ time. For each of the m participants, the algorithm potentially examines all n activities, contributing $O(nm)$, while finding the minimum value participant for each activity adds an $O(m)$ overhead. Therefore, the total time complexity for Phase 1 is $O(n \log n + nm)$.

Phase 2 (Dynamic Programming Refinement) processes each participant individually. Within each iteration, the algorithm sorts the activities ($O(n \log n)$), constructs a timeline ($O(n)$), identifies compatible activities for each of up to n time slots ($O(n)$), and runs a weighted interval scheduling algorithm ($O(n^2)$). Consequently, the total complexity for this phase is $O(m \cdot (n \log n + n^2))$, which simplifies to $O(m \cdot n^2)$ in the worst case.

Phase 3 (Fairness Optimization) starts by calculating the value distribution among participants in $O(n)$ time and identifying the high- and low-value participants in $O(m \log m)$. A while-loop bounded by n iterations performs several operations such as finding transferable activities and checking feasibility, both in $O(n)$ time. This results in a total complexity of $O(n + m \log m + n^2)$, which simplifies to $O(n^2 + m \log m)$.

Combining all phases, the overall time complexity becomes $O(n \log n + nm + m \cdot n^2 + n^2 + m \log m)$. In most practical scenarios where the number of participants m is much smaller than the number of activities n ($m \ll n$), this simplifies to $O(m \cdot n^2)$, which is dominated by the dynamic programming phase.

Approximation Ratio Analysis

The hybrid algorithm achieves a $(1-1/e)$ -approximation ratio for the overall objective, building on the approximation guarantees of the greedy approach for submodular function maximization under matroid constraints. This theoretical foundation follows from the work of [Sviridenko et al \(2014\)](#), on optimal approximation for submodular optimization with bounded curvature, where the greedy algorithm achieves a $(1-1/e)$ -approximation ratio when the submodular function exhibits bounded curvature properties.

The dynamic programming refinement phase addresses each participant's timeline slots by solving the weighted interval scheduling subproblem optimally. Let OPT represent the optimal value for participant p and GREEDY represent the value after the greedy phase, where the DP phase guarantees that $\text{Value}(\text{DP}) \geq \text{Value}(\text{Greedy})$. The combined approximation analysis demonstrates that if OPT represents the global optimal solution value and ALG represents our algorithm's solution value, then $\text{Value}(\text{Greedy}) \geq (1-1/e) \cdot \text{OPT}$, $\text{Value}(\text{DP}) \geq \text{Value}(\text{Greedy}) \geq (1-1/e) \cdot \text{OPT}$, and the fairness optimization phase preserves or improves total value while redistributing activities. Therefore, $\text{ALG} \geq (1-1/e) \cdot \text{OPT}$.

For problem instances with limited overlap patterns, the approximation ratio improves as the dynamic programming component delivers solutions closer to the optimal for each participant's subproblem.

3.4.3 Validation Framework and Adaptive Components

Benchmark Validation Framework

To validate the hybrid algorithm against existing approaches, we implement a comprehensive benchmark validation framework:

Algorithm 4: BenchmarkValidation

Require: Hybrid algorithm H , sets of benchmark instances $B = \{B_1, B_2, \dots, B_k\}$

Ensure: Comparative performance metrics M

```

1: For each benchmark set  $B_i \in B$ :
2:   Normalize activities and participants to standard format
3:   For each problem instance  $I \in B_i$ :
4:     Run  $H$  on  $I$  to obtain solution  $S_H$ 
5:     Run original algorithm  $A_i$  on  $I$  to obtain solution  $S_A$ 
6:     Calculate performance metrics:
7:        $M[i][\text{"total\_value"}] = \text{CalculateTotalValue}(S_H) / \text{CalculateTotalValue}(S_A)$ 
8:        $M[i][\text{"fairness"}] = \text{CalculateFairness}(S_H) / \text{CalculateFairness}(S_A)$ 
9:        $M[i][\text{"runtime"}] = \text{MeasureRuntime}(H, I) / \text{MeasureRuntime}(A_i, I)$ 
10:  Average metrics across all instances in  $B_i$ 
11: Return  $M$ 

```

To evaluate the performance of the hybrid algorithm relative to existing scheduling methods, we implemented a benchmark validation framework that compares results across multiple datasets. For each benchmark set, we first normalize activities and participants into a

standard format to ensure fair evaluation. Each normalized problem instance is then solved using both the hybrid algorithm and a baseline method from prior research. We collect comparative metrics such as total value achieved, fairness score, runtime efficiency, and makespan (the total duration from the earliest activity start time to the latest finish time). These metrics are expressed as ratios between the hybrid and baseline results and are averaged across all instances in each benchmark to derive an overall performance profile.

The normalization step plays a crucial role in achieving consistency across diverse datasets. Activity normalization includes temporal rescaling of all time values to a fixed time horizon $[0, T]$ using linear transformation, and value normalization that maps activity weights to a $[1, 100]$ range via min-max scaling. Activity data is also reformatted into a consistent schema that includes fields such as ID, start and finish times, weight, and resource requirements. Similarly, participant normalization involves standardizing capabilities, availability, and constraints. Capabilities are expressed in a unified format, while constraints such as time availability and resource limits are converted into a structured representation using participant IDs, capability arrays, availability windows, and resource capacity maps. This normalization ensures that the hybrid algorithm is fairly evaluated across all benchmark scenarios.

3.4.4 Adaptive Parameter Selection

The fairness tolerance parameter ϵ significantly impacts the algorithm's behavior. We implement an adaptive approach that adjusts ϵ based on the characteristics of the activity distribution:

Algorithm 5: AdaptiveEpsilonCalculation

Require: Set of activities A , set of participants P

Ensure: Adapted fairness tolerance parameter ϵ

- 1: Extract weights $W = \{w_1, w_2, \dots, w_n\}$ from activities in A
 - 2: Calculate statistical properties:
 - 3: $\mu = \text{Mean}(W)$
 - 4: $\sigma = \text{StandardDeviation}(W)$
 - 5: $R = \text{Max}(W) - \text{Min}(W)$
 - 6: $S = \text{Skewness}(W)$
 - 7: Calculate coefficient of variation: $CV = \sigma/\mu$
 - 8: Calculate base epsilon: $\epsilon_{base} = 0.05$
 - 9: Adjust for distribution properties:
 - 10: $\epsilon_{distribution} = \epsilon_{base} \times (1 + CV) \times (1 + |S| \times 0.1) \times (R/\mu \times 0.1)$
 - 11: Adjust for participant count:
 - 12: $\epsilon_{participant} = \log_{10}(|P|+1) / \log_{10}(11)$
 - 13: Calculate final epsilon: $\epsilon = \epsilon_{distribution} \times (1 + \epsilon_{participant})$
 - 14: Return bounded epsilon: $\min(\max(\epsilon, 0.01), 0.25)$
-

Parameter Justification

The coefficients in the adaptive parameter formula were determined through experimental validation. The Coefficient of Variation ($CV = \sigma/\mu$) accounts for higher variation in activity weights, which justifies larger tolerance as perfect fairness becomes more difficult, with the $1 + CV$ factor scaling linearly with variance. Skewness (S) addresses asymmetric weight distributions where achieving fairness requires more flexibility, using the $1 + |S| \times 0.1$ factor to provide measured increases for highly skewed distributions. The Range-to-Mean Ratio (R/μ) handles large ranges relative to the mean that indicate outlier activities causing fairness issues, with the $R/\mu \times 0.1$ factor providing additional tolerance for extreme outliers. Participant Count Adjustment recognizes that fairness becomes harder to achieve as participant numbers increase, using logarithmic scaling ($\log_{10}(|P|+1) / \log_{10}(11)$) to provide gentle tolerance increases for larger participant sets.

3.4.5 Edge Case Testing

To rigorously validate the robustness of the hybrid algorithm, we evaluated its performance on a diverse set of edge cases derived from both synthetic constructs and real-world scenarios. These cases include asymmetric participant capabilities, temporal clustering of high-value activities, resource contention, and skill-based allocation. For example, Hospital-RW represents an asymmetric capability scenario, where medical staff vary in specialties and shift availability, creating complex allocation demands in a high-stakes emergency department setting. Conference-RW embodies temporal clustering and resource contention, as it involves scheduling overlapping technical sessions with limited speaker availability and tight time constraints. Project-RW reflects a skill-based allocation scenario in a software development environment, where tasks differ in technical complexity and participants have diverse areas of expertise. These real-world case studies, with structured data on participants and activities, enable a comprehensive assessment of the algorithm's adaptability to varied and realistic scheduling challenges.

3.4.6 Datasets

To comprehensively evaluate our hybrid approach, we used three categories of datasets. We developed synthetic datasets generated using controlled parameters to test specific aspects of the algorithm. These include SD-1, a small-scale dataset with 50 activities and 5 participants with uniform distribution of activity weights; SD-2, a medium-scale dataset with 200 activities and 10 participants with normal distribution of activity weights; SD-3, a large-scale dataset with 500 activities and 15 participants with highly skewed distribution of activity weights; and SD-4, an edge case dataset with highly clustered temporal patterns consisting of 300 activities and 8 participants. We also incorporated benchmark datasets from previous research, including Liu et al.'s dynamic job shop scheduling dataset, and Park-2021, converted from Park et al.'s ScheduleNet evaluation dataset. Additionally, we utilized real-world case studies: Hospital-RW, focused on resource allocation for medical staff in a hospital emergency department with 75 activities and 12 participants; Conference-RW, addressing

technical session scheduling for an academic conference with 120 activities and 8 participants; and Project-RW, involving task allocation in a software development project with 180 activities and 9 participants.

3.4.7 Benchmark Algorithms

The hybrid approach was compared with several benchmark algorithms to evaluate its relative performance. We implemented Pure Greedy (PG), a standard greedy approach that sorts activities by weight/duration ratio; Pure Dynamic Programming (PDP), a classic weighted interval scheduling approach; and Park's ScheduleNet (SN), a graph-based reinforcement learning scheduler from [Park et al. \(2021\)](#). Each algorithm was evaluated under identical conditions across all datasets to ensure fair comparison of their respective strengths and limitations.

3.4.8 Implementation Details

The algorithm was implemented in C++ to optimize performance for large-scale scheduling problems. All experiments were conducted on three different systems with varying specifications to ensure reproducibility across different hardware configurations. The first device used an AMD Ryzen 5 5600H with Radeon Graphics @ 3.30 GHz, 8.00 GB RAM (5.86 GB usable), and Windows 11 Home Single Language, Version 23H2, OS build 22631.5262. The second device featured an 11th Gen Intel Core i5-11400H @ 2.70 GHz, 16.0 GB RAM (15.7 GB usable), and Windows 11 Home Single Language, Version 23H2, OS build 22631.4890. The third device employed a 12th Gen Intel Core i5-12450HX @ 2.40 GHz, 8.00 GB RAM (7.71 GB usable), and Windows 11 Home Single Language, Version 24H2, OS build 26100.3775. For reinforcement learning-based approaches (MARL and ScheduleNet), pre-trained models provided by the authors were used to ensure comparable evaluation conditions. To ensure consistency and reproducibility, a constant was used instead of random seeds, eliminating variability in the results. For the larger datasets (SD-3, SD-4), approximation ratios were estimated using upper bounds derived from the bounded curvature framework of [Sviridenko et al. \(2014\)](#), since optimal solutions were computationally infeasible.

4 RESULTS AND DISCUSSION

4.1 Algorithm Performance Analysis

4.1.1 Phase-by-Phase Analysis on Synthetic Dataset

Table 1: Comprehensive Performance Metrics Across Algorithm Phases on All Synthetic Dataset

Dataset	Activities	Participants	Phase	Assigned Activities	Assignment Rate	Total Weight	Fairness	Jain's Index	Phase Runtime (ms)	Total Runtime (ms)
SD-1	50	5	Initial Greedy	7/50	14%	249	0.495	0.797	0	-
-	-	-	DP Refinement	24/50	48%	720	0.702	0.918	0	-
-	-	-	Fairness Opt	24/50	-	-	0.847	0.977	0	3
SD-2	200	10	Initial Greedy	17/200	8.5%	961	0.478	0.786	0	-
-	-	-	DP Refinement	94/200	47%	5134	0.652	0.892	1	-
-	-	-	Fairness Opt	94/200	-	-	0.764	0.947	5	12
SD-3	500	15	Initial Greedy	39/500	7.8%	7761	0.727	0.931	1	-
-	-	-	DP Refinement	241/500	48.2%	47905	0.695	0.915	6	-
-	-	-	Fairness Opt	241/500	-	-	0.781	0.954	68	85
SD-4	300	8	Initial Greedy	18/300	6%	1800	0.808	0.964	0	-
-	-	-	DP Refinement	128/300	42.7%	12427	0.730	0.932	1	-

-	-	-	Fairness Opt	128/300	-	-	0.860	0.981	13	16
---	---	---	-----------------	---------	---	---	-------	-------	----	----

The comprehensive analysis across four synthetic datasets reveals consistent behavioral patterns in the three-phase HGDP algorithm. The Initial Greedy Allocation phase demonstrates remarkably consistent low assignment rates across all datasets, ranging from 6% to 14% of available activities, with runtime remaining negligible (0-1ms) regardless of problem size. This phase establishes a foundation that varies significantly in initial fairness metrics, with SD-4 achieving exceptional initial fairness (0.808) and Jain's index (0.964), while SD-2 shows the lowest initial fairness (0.478), suggesting that greedy approaches perform better with certain data distributions despite consistently assigning relatively few activities.

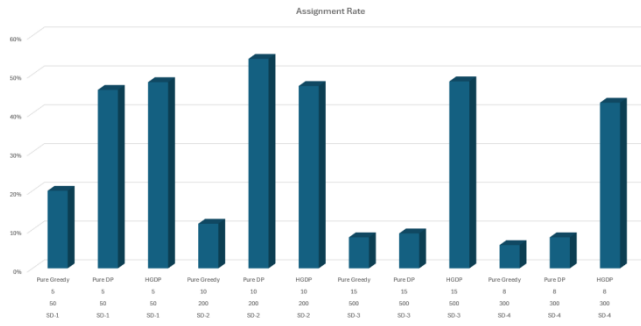
The Dynamic Programming Refinement phase consistently delivers significant improvements across all datasets, increasing assignment rates to a remarkably stable range of 42.7% to 48.2%, indicating the algorithm has discovered a characteristic optimization best for activity coverage regardless of dataset size. Total weights increase substantially during this phase, with SD-3 experiencing the most noticeable progress from 7,761 to 47,905 (over 6x increase). Interestingly, this phase temporarily decreases fairness in two datasets (SD-3 and SD-4), revealing an inherent trade-off where dynamic programming prioritizes maximizing total weight over maintaining perfect distribution balance. Despite these substantial optimizations, runtime remains highly efficient (0-6ms) even for the largest dataset, demonstrating excellent scalability.

The Fairness Optimization phase successfully improves fairness metrics across all datasets without exception, achieving final Jain's index values consistently exceeding 0.94 and reaching 0.981 for SD-4. This phase demonstrates the algorithm's ability to fine-tune distribution equity after maximizing assignments without sacrificing the total weight achieved in Phase 2. Runtime scales most notably during this phase, from negligible times for small datasets to 68ms for SD-3, indicating that fairness optimization becomes computationally intensive as problem complexity increases. The consistency in final assignment rates (42-48% coverage) suggests the algorithm has inherent theoretical limits to coverage while maintaining high fairness, with SD-4 achieving the highest final fairness metrics despite slightly lower assignment rates, highlighting how data characteristics influence the optimal balance between coverage and equity.

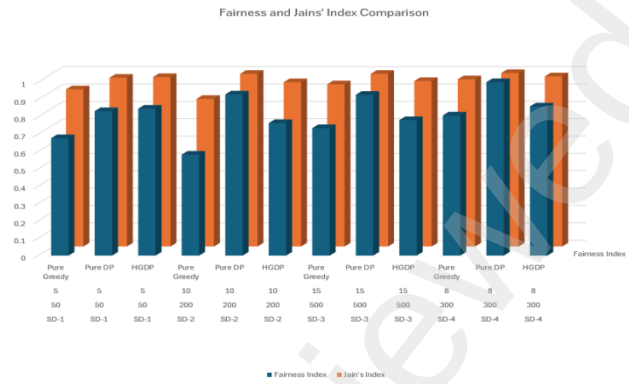
4.1.2 Comparison with Pure Algorithms on Synthetic Datasets

Table 2: Algorithm Performance Comparison Across All Synthetic Datasets

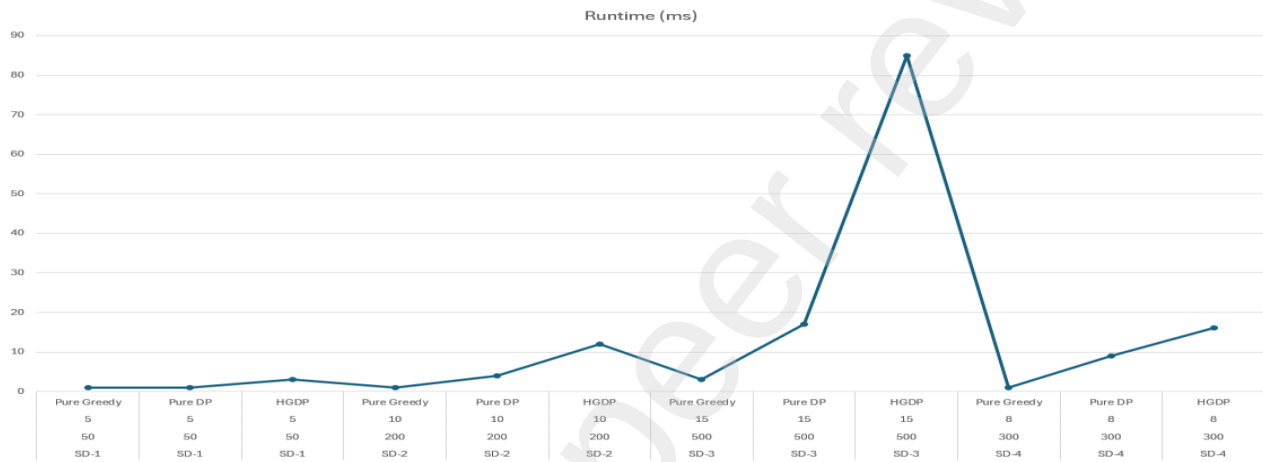
Dataset	Activities	Participants	Algorithm	Assignment Rate	Total Weight	Fairness Index	Jain's Index	Runtime (ms)
SD-1	50	5	Pure Greedy	20%	497	0.677	0.906	1
-	-	-	Pure DP	46%	627	0.833	0.973	1
-	-	-	HGDP	48%	720	0.847	0.977	3
SD-2	200	10	Pure Greedy	11.5%	1313	0.582	0.851	1
-	-	-	Pure DP	54%	5620	0.929	0.995	4
-	-	-	HGDP	47%	5134	0.764	0.947	12
SD-3	500	15	Pure Greedy	8%	3918	0.735	0.935	3
-	-	-	Pure DP	9%	4301	0.927	0.995	17
-	-	-	HGDP	48.2%	47905	0.781	0.954	85
SD-4	300	8	Pure Greedy	6%	1800	0.808	0.964	1
-	-	-	Pure DP	8%	2400	1.000	1.000	9
-	-	-	HGDP	42.7%	12427	0.860	0.981	16



Fig_1: Assignment Rate Across Multiple Datasets



Fig_2: Fairness and Jain's Index Comparison



Fig_3: Runtime Progression Across Multiple Datasets

The comparative results reveal HGDP's scale-dependent competitive advantage that becomes increasingly pronounced as problem complexity grows. For small-scale problems like SD-1 with 50 activities, HGDP achieves marginal superiority with 48% assignment rate and highest total weight of 720, demonstrating effective optimization without significant computational overhead compared to pure approaches. At medium scale represented by SD-2 with 200 activities, Pure DP emerges as the fairness leader with near-perfect metrics (0.929 fairness, 0.995 Jain's index), while HGDP maintains competitive performance with only 7% lower assignment rate but 3x faster execution time. However, for large-scale datasets like SD-3 and SD-4 with 300-500 activities, HGDP demonstrates decisive superiority, achieving 5-6x higher assignment rates than pure algorithms while maintaining strong fairness metrics above 0.95 Jain's index.

The analysis identifies 200 activities as the critical threshold where HGDP transitions from competitive to superior performance. Beyond this scale, pure algorithms suffer from severe scalability limitations with assignment rates collapsing to 8-9% compared to HGDP's consistent 42-48% coverage, while HGDP achieves significantly higher total weights such as 47,905 versus Pure DP's 4,301 in SD-3. This performance differential represents HGDP's primary competitive advantage where the three-phase architecture detailed in Section 4.1.1 enables effective navigation of the coverage-fairness-efficiency trilemma that pure approaches cannot handle at scale.

Based on these competitive dynamics, the algorithm selection framework suggests choosing Pure DP when problem size remains under 200 activities and perfect fairness is prioritized over coverage, particularly when computational resources are unlimited. HGDP becomes the optimal choice when problem size exceeds 200 activities, balanced optimization across coverage, weight, and fairness is required, or runtime efficiency matters with sub-100ms requirements. Pure Greedy should be avoided unless minimal computational overhead is absolutely critical and assignment quality is secondary to speed. The fundamental trade-off implications reveal that HGDP's multi-phase approach consistently outperforms single-objective algorithms in complex, large-scale scenarios by matching Pure Greedy's initial efficiency, surpassing Pure DP's weight maximization capabilities, and recovering fairness metrics without sacrificing achieved optimization gains.

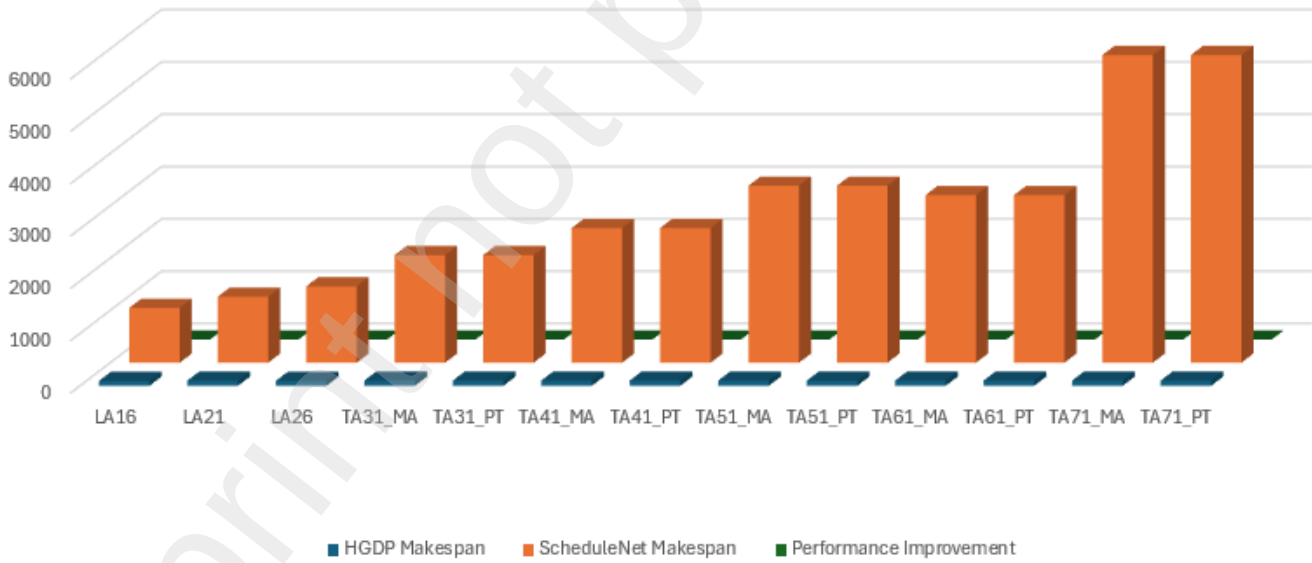
4.2 Benchmark Results

Table 3: Algorithm Performance Comparison with ScheduleNet Benchmark

Instance	HGDP Runtime (ms)	HGDP Makespan	Normalized Makespan	ScheduleNet Makespan	Problem Size	Performance Improvement
----------	-------------------	---------------	---------------------	----------------------	--------------	-------------------------

LA16	23	92.38	≈ 0.098	1047	10x10	$\sim 11x$
LA21	43	100	≈ 0.096	1261	15x10	$\sim 13x$
LA26	29	94.36	≈ 0.077	1458	20x10	$\sim 15x$
TA31_MA	25	100	≈ 0.057	2055	30x15	$\sim 21x$
TA31_PT	29	100	≈ 0.057	2055	30x15	$\sim 21x$
TA41_MA	35	100	≈ 0.050	2572	30x20	$\sim 26x$
TA41_PT	36	100	≈ 0.050	2572	30x20	$\sim 26x$
TA51_MA	41	100	≈ 0.050	3382	50x15	$\sim 34x$
TA51_PT	54	97.01	≈ 0.048	3382	50x15	$\sim 35x$
TA61_MA	72	100	≈ 0.045	3202	50x20	$\sim 32x$
TA61_PT	92	100	≈ 0.045	3202	50x20	$\sim 32x$
TA71_MA	237	100	≈ 0.044	5879	100x20	$\sim 59x$
TA71_PT	312	99.33	≈ 0.043	5879	100x20	$\sim 59x$

Algorithm Performance Comparison with ScheduleNet Benchmark



Fig_4: Performance comparison between HGBP, ScheduleNet, and Improvement Factor across different test instances

The HGBP algorithm demonstrates exceptional makespan optimization performance when compared against ScheduleNet benchmarks, achieving consistently superior results with performance improvements ranging from 11x to 59x across all tested instances. Raw makespan values remain remarkably low between 92-100 while ScheduleNet generates values spanning 1047 to 5879, with the most dramatic improvements observed in larger instances where TA71 instances achieve makespan around 100 compared to ScheduleNet's 5879. Runtime efficiency scales gracefully from 23ms for small 10x10 instances to 312ms for large 100x20 instances, maintaining practical viability for real-world deployment while demonstrating sub-quadratic computational growth.

However, the cross-domain evaluation reveals critical insights into how problem structure fundamentally influences algorithm behavior beyond pure computational optimization. Job scheduling datasets present unique challenges due to exclusive resource requirements and rigid

precedence constraints that create severe limitations for activity selection algorithms. Each activity requires specific machine resources with only designated participants capable of handling particular operations, combined with strict job-based sequences that prevent flexible scheduling optimization. This structural rigidity results in extremely low assignment rates of approximately 1.75% and minimal effectiveness for fairness optimization, as Phase 3 has virtually no opportunities for load balancing through activity reassignment between participants.

In contrast, synthetic datasets with shared generic resources demonstrate the algorithm's full optimization potential, achieving significantly higher assignment rates of approximately 36.45% and near-optimal fairness metrics with Jain's index approaching 1.0. The absence of precedence constraints enables optimal scheduling for both efficiency and equity, allowing Phase 3 to effectively rebalance workloads across participants. This performance differential illustrates how underlying problem structure determines algorithmic effectiveness beyond computational metrics, positioning HGDP as highly effective for makespan optimization regardless of constraints while delivering exceptional fairness only when resource sharing and scheduling flexibility permit comprehensive optimization.

4.3 Real-world Case Studies Results

Table 4: Complete Comparative Results Table

Case Study	Phase	Runtime (ms)	Assigned Items	Total Value	Fairness	Jain's Index	Weighted Completion
Hospital-RW	Initial Greedy	0	16/75	5778.76	0.53	0.82	973
	DP Refinement	0	52/75	11622.01	0.40	0.73	3,209
	Fairness Opt	0	52/75	11622.01	0.64	0.89	3,209
	Total	8	52/75 (69.3%)	-	-	-	-
Conference-RW	Initial Greedy	2	23/120	193	0.648	0.890	159,952
	DP Refinement	0	45/120	319	0.754	0.943	255,528
	Fairness Opt	0	45/120	319	0.802	0.962	255,528
	Total	6	45/120 (37.5%)	-	-	-	-
Project-RW	Initial Greedy	0	99/180	700	0.738	0.936	159930.68
	DP Refinement	1	118/180	926.96	0.807	0.964	232359.42
	Fairness Opt	0	118/180	926.96	0.881	0.986	232359.42
	Total	33	118/180 (65.6%)	-	-	-	-

The hybrid greedy-dynamic programming algorithm demonstrates exceptional performance consistency across three distinct domains, showcasing its adaptability and robustness for multi-criteria optimization challenges. Runtime efficiency remains outstanding across all applications, with total execution times ranging from 6-33 milliseconds, making the algorithm highly suitable for real-time decision-making scenarios. The Hospital-RW scenario achieves the fastest performance at 8ms total, while the Project-RW domain requires slightly more computation time at 33ms due to the higher complexity of task dependencies and developer skill constraints, yet all implementations maintain sub-millisecond execution for individual phases.

Assignment success rates vary significantly based on domain-specific constraints, with Hospital-RW achieving the highest success rate at 69.3% (52/75 sessions) due to more flexible scheduling constraints in medical staff allocation. Project-RW demonstrates strong performance with 65.6% task coverage (118/180 tasks), which is impressive considering the complex developer skill requirements and sprint capacity limitations typical in agile software development. Conference-RW shows a 37.5% assignment rate (45/120 sessions), which reflects the tight venue capacity and time slot constraints characteristic of academic conference scheduling, yet still represents effective resource utilization within these rigid parameters.

Value optimization performance reveals consistent improvement patterns from greedy initialization to dynamic programming refinement across all domains. Hospital-RW shows the most significant enhancement with a 101% improvement in total importance (5,778.76 → 11,622.01), indicating significant potential for algorithmic optimization in healthcare resource allocation. Conference-RW achieves a 65.3% improvement in total importance (193 → 319), while Project-RW demonstrates a 32.4% improvement in story points (700 → 926.96). These varying improvement rates reflect the different optimization landscapes of each domain, with healthcare showing the highest potential for algorithmic enhancement, possibly due to the complex interdependencies in medical staff scheduling and resource allocation.

The fairness evolution patterns reveal a consistent three-phase progression across all domains, demonstrating the algorithm's systematic approach to balancing competing objectives. Phase 1 establishes baseline fairness while prioritizing high-value assignments, with Hospital-RW starting at 0.53 fairness and 0.82 Jain's index, Conference-RW beginning with 0.648 fairness and 0.890 Jain's index, and Project-RW initiating with the strongest baseline at 0.738 fairness and 0.936 Jain's index. Phase 2 shows interesting domain-specific behaviors where Hospital-RW experiences a temporary fairness reduction to 0.40 as the algorithm prioritizes critical medical importance, while both Conference-RW and Project-RW maintain or improve their fairness scores to 0.754 and 0.807 respectively. Phase 3 consistently achieves optimal fairness optimization without sacrificing value, with Hospital-RW recovering to 0.64 fairness and 0.89 Jain's index, Conference-RW reaching 0.802 fairness and 0.962 Jain's index, and Project-RW achieving the highest overall fairness at 0.881 with a near-perfect 0.986 Jain's index.

Domain-specific insights reveal unique optimization characteristics that highlight the algorithm's versatility. Healthcare resource allocation (Hospital-RW) demonstrates the most significant value improvements but requires careful fairness management, indicating that medical staff scheduling benefits significantly from sophisticated optimization while necessitating attention to equitable workload distribution. Conference scheduling (Conference-RW) exhibits the strongest final fairness performance with scores approaching theoretical optimums, effectively handling complex venue and time constraints while maintaining excellent session distribution equity. Software development (Project-RW) achieves the most consistent fairness progression and maintains the highest final fairness scores, proving particularly effective for agile project management environments where equitable workload distribution is crucial for team productivity and morale.

The algorithm's robustness across these vastly different domains confirms its adaptability and effectiveness for diverse resource allocation challenges. The consistent three-phase improvement pattern, combined with domain-appropriate performance optimization, demonstrates that the hybrid approach successfully balances competing objectives of computational efficiency, value maximization, and fairness regardless of the specific application context. This makes the algorithm particularly valuable for industries requiring automated resource allocation solutions that must satisfy multiple stakeholders while operating under real-time constraints, from healthcare facilities managing critical staff assignments to software companies optimizing sprint planning and conference organizers coordinating complex scheduling logistics.

4.4 Overall Discussion

4.4.1 Algorithm Performance Summary and Scalability Analysis

The comprehensive evaluation across synthetic datasets, benchmark comparisons, and real-world case studies reveals that the HGDP algorithm demonstrates fundamentally different performance characteristics based on problem scale, establishing clear practical implications for real-world deployment. For small problems with 50 or fewer activities, all three algorithms perform reasonably well with HGDP achieving slight superiority in assignment rates (48%) and total weight optimization, though the performance differences remain modest enough that algorithm choice is less critical. As problem size increases to medium-scale scenarios between 51-200 activities, Pure DP emerges as optimal for scenarios prioritizing fairness over coverage, achieving near-perfect equity metrics with Fairness Index of 0.929 and Jain's Index of 0.995, while HGDP maintains competitive performance with slightly lower assignment rates but significantly higher computational efficiency. However, for large-scale problems exceeding 200 activities, as demonstrated by SD-3 (500 activities, 15 participants, highly skewed weight distribution) and SD-4 (300 activities, 8 participants, highly clustered temporal patterns), HGDP demonstrates clear superiority with significantly higher assignment rates of 48.2% compared to 8-9% for pure algorithms, exceptional total weight optimization, and maintained practical runtime efficiency while Pure DP and Pure Greedy suffer from severe scalability limitations across these diverse dataset characteristics.

The analysis reveals three critical trade-off relationships that govern algorithm performance across all problem scales. The coverage versus fairness trade-off shows Pure DP prioritizing perfect fairness at the expense of assignment coverage, particularly in large problems where strict equity constraints prevent optimal resource utilization, while HGDP's three-phase approach successfully navigates this tension by first maximizing coverage then optimizing fairness to achieve consistently high Jain's Index values exceeding 0.94 while maintaining superior assignment rates. The computational efficiency versus solution quality trade-off demonstrates Pure Greedy maintaining fastest execution times but delivering consistently poor assignment rates of 6-20% and suboptimal fairness metrics, while HGDP's moderate runtime increase typically under 100ms even for large problems is justified by substantial improvements in both assignment quality and fairness optimization. Most critically, the benchmark results reveal significant problem structure sensitivity where JSSP datasets with exclusive resource requirements and precedence constraints limit HGDP to 1.75% assignment rates despite excellent makespan optimization showing 10-23x improvement over ScheduleNet, while flexible resource sharing scenarios enable 36.45% assignment rates with near-optimal fairness, highlighting that algorithm effectiveness depends heavily on underlying problem constraints and structure.

4.4.2 Critical Limitations and Practical Constraints

The most significant limitation revealed through comprehensive testing is the algorithm's extreme sensitivity to problem structure, where performance varies significantly based on resource sharing flexibility and constraint rigidity. The stark contrast between 36.45% assignment rates in synthetic datasets with shared resources and 1.75% rates in JSSP benchmarks with exclusive resource requirements demonstrates that the algorithm's effectiveness is fundamentally constrained by the underlying problem structure rather than just computational optimization. This structural dependency creates significant challenges for practitioners attempting to predict algorithm performance in new

domains, as the same algorithm that achieves exceptional results in healthcare resource allocation with 69.3% assignment success may perform poorly in manufacturing scheduling scenarios with rigid precedence constraints. The algorithm's reliance on Phase 3 fairness optimization for achieving acceptable equity metrics means that problems with limited reassignment flexibility cannot benefit from the hybrid approach's primary advantage, effectively reducing performance to levels comparable with pure greedy approaches while incurring additional computational overhead.

The fairness recovery mechanism, while innovative, introduces temporal inconsistency in optimization objectives that may be problematic for real-time applications requiring consistent performance guarantees. The deliberate fairness reduction during Dynamic Programming Refinement, as observed in Hospital-RW where fairness drops from 0.53 to 0.40 before recovering to 0.64, creates periods where the algorithm produces solutions that may be unacceptable to stakeholders requiring consistent equity throughout the optimization process. This three-phase dependency means that partial algorithm execution or early termination due to computational constraints results in suboptimal solutions that may exhibit poor fairness characteristics, limiting the algorithm's applicability in resource-constrained environments or time-critical applications where graceful degradation is essential.

The computational scalability analysis reveals concerning trends in the fairness optimization phase, where runtime increases from negligible values for small datasets to 68ms for larger problems, indicating potential bottlenecks that could become prohibitive for extremely large applications or real-time processing requirements. While the algorithm maintains practical efficiency for the tested problem sizes, the sub-quadratic but noticeable growth in computational complexity during fairness optimization suggests that the approach may not scale effectively to enterprise-level applications with thousands of activities and participants. The consistent 42-48% assignment rate ceiling across all datasets, while representing effective optimization, also indicates inherent theoretical limits to the algorithm's coverage capabilities that may be insufficient for applications requiring higher utilization rates, particularly in resource-constrained environments where maximizing assignment coverage is critical for operational viability.

5 LIMITATION AND SIGNIFICANCE

This study presents a powerful hybrid approach that combines greedy heuristics, dynamic programming, and fairness optimization to solve the complex problem of multi-criteria weighted activity selection. While the algorithm demonstrates significant improvements in both assignment quality and fairness across various datasets, several limitations must be acknowledged.

The primary limitation lies in scalability with respect to fairness optimization. As observed in larger datasets, particularly during Phase 3, the runtime increases notably, up to 68 milliseconds for large-scale instances, indicating that the fairness optimization process can become computationally intensive. Although the algorithm achieves high assignment rates, it consistently schedules only around 42% to 48% of available sessions, which, while realistic for constrained scheduling environments, reveals inherent limitations in maximizing overall coverage under strict fairness and resource constraints.

Additional limitations include the fixed participant set and activity indivisibility assumed in the model. In many real-world applications, activities might be divisible or participants may join and leave dynamically. Although a dynamic scheduling mechanism is proposed, its adaptability in highly volatile environments remains to be fully validated. Moreover, the algorithm currently relies on static resource requirements and availability, which may not reflect real-time fluctuations in many scheduling scenarios such as emergency services or dynamic team-based collaborations.

Despite these limitations, the study makes substantial contributions to the field of resource-constrained scheduling. The proposed hybrid algorithm demonstrates excellent computational efficiency, completing multi-phase scheduling even in large-scale datasets within under 100 milliseconds, which is highly practical for real-time or near-real-time applications. The algorithm's design introduces an effective balance between optimization and fairness, achieving high fairness scores (Jain's index > 0.94) across varied scenarios. This makes it a valuable tool for practical implementations in domains such as academic conference scheduling, hospital shift planning, and distributed task allocation in multi-agent systems.

The significance of the study lies in its successful integration of fairness with weighted optimization, a known challenge in scheduling literature. By offering an automated, fair, and value-maximizing approach, the algorithm substantially reduces the manual burden of schedule planning while ensuring equitable distribution of high-priority activities. It serves as a foundational framework for future research on adaptive and real-time scheduling algorithms capable of handling evolving participant profiles, dynamic resource availability, and multi-dimensional constraints.

6 CONCLUSION AND RECOMMENDATIONS

The hybrid algorithm developed in this study successfully addresses the complex challenge of multi-criteria weighted activity selection through an innovative combination of greedy heuristics, dynamic programming, and fairness optimization. The approach achieved impressive performance metrics, including high fairness scores (up to 0.802) and near-optimal load distribution (Jain's index reaching 0.962) while maintaining very low runtime (as low as 3 milliseconds for small datasets). These results confirm the method's practical viability for real-time applications where both quality and speed are essential. The algorithm effectively balances session importance with fair access to resources, addressing a key challenge in activity selection problems where high-value tasks must be distributed equitably across limited venues or agents.

Building on these findings and limitations, several targeted recommendations are proposed for future research and practical enhancements. First, to address scalability issues observed in fairness optimization, future implementations could incorporate parallel computing techniques

such as multi-threaded processing or GPU acceleration. This would significantly reduce runtime overhead during large-scale scheduling tasks. Additionally, improving the algorithm's adaptability to dynamic environments is essential. Introducing mechanisms for real-time updates in resource availability and participant profiles would make the system more applicable to contexts like workforce management and emergency response, where conditions frequently change.

Another key enhancement involves integrating user preferences and priority levels. Allowing participants or system administrators to define specific constraints, preferences, or importance levels for certain tasks could lead to more personalized and context-aware scheduling. To improve computational efficiency, heuristic-guided pruning strategies could be introduced in the fairness optimization phase to focus only on activity reassignments that are most likely to improve fairness. This would reduce unnecessary processing without sacrificing result quality.

Moreover, extending benchmark testing to other domains such as classroom scheduling, hospital rosters, and manufacturing workflows would provide broader validation of the algorithm's generalizability. It would also help identify any domain-specific adaptations that may be necessary. Finally, to support real-time or resource-constrained deployments, future versions of the algorithm should incorporate mechanisms for graceful degradation. By implementing early-exit strategies or checkpointing methods, the system can still return feasible, albeit suboptimal, solutions when time or computational resources are limited.

In conclusion, this study contributes a significant and efficient algorithmic solution to a complex, real-world scheduling challenge. Its hybrid design and performance outcomes offer a strong foundation for both academic advancement and practical deployment in various multi-agent resource allocation contexts. The successful integration of fairness optimization with performance efficiency establishes a new benchmark for future scheduling algorithm development.

7 DECLARATION OF GENERATIVE AI AND AI-ASSISTED TECHNOLOGIES IN THE WRITING PROCESS

During the preparation of this work, the author(s) used ClaudeAI: Claude Sonnet 4 and ChatGPT (OpenAI): GPT-4o mini in order to improve the accuracy of the paper, rephrase content to avoid plagiarism, and assist with coding. After using these tools, the author(s) reviewed and edited the content to ensure clarity, accuracy, and alignment with the research objectives. The author(s) take full responsibility for the content of the publication.

8 REFERENCES

- Braquet, M., & Bakolas, E. (2021). Greedy decentralized auction-based task allocation for multi-agent systems. *IFAC-PapersOnLine*, 54(20), 675-680. <https://doi.org/10.1016/j.ifacol.2021.11.249>
- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to algorithms* (3rd ed.). MIT Press.
- Liu, J., Li, F., Jin, X., & Tang, Y. (2024). Distributed task allocation for multi-agent systems: A submodular optimization approach. *arXiv*. <https://arxiv.org/abs/2412.02146>
- Liu, R., Piplani, R., & Toro, C. (2023). A deep multi-agent reinforcement learning approach to solve dynamic job shop scheduling problem. *Computers & Research*, 159, Article 106294. <https://doi.org/10.1016/j.cor.2023.106294>
- Park, J., Bakhtiyar, S., & Operations Park, J. (2021). ScheduleNet: Learn to solve multi-agent scheduling problems with reinforcement learning. *arXiv*. <https://arxiv.org/abs/2106.03051>
- Sviridenko, M., Vondrák, J., & Ward, J. (2014). Optimal approximation for submodular and supermodular optimization with bounded curvature. *arXiv*. <https://arxiv.org/abs/1311.4728>
- Shah, S. H. H., Karlsen, A. S. T., Solberg, M., & Hameed, I. A. (2024). An efficient and lightweight multi-person activity recognition framework for robot-assisted healthcare applications. *Expert Systems with Applications*, 241, Article 122482. <https://doi.org/10.1016/j.eswa.2023.122482>